Aryan Agarwal aab5473, Hirsch Prasad hkp423
Dr. August Shi
ECE382V Software Testing in the Nondeterministic Era
22 October 2026

Checkpoint 1

Answering Proposal Feedback

1. **What are your planned RQs and what metrics would you use to answer the RQs? For example, you mention checking whether perturbations follow documented rules, but how exactly are you going to be checking that? What would you measure in that case?**

   Our RQ: How can we use LLMs to automatically identify applications of API calls that do not account for the API's non-deterministic nature, and can we identify more bugs than leading systems (NonDex)?

   We would be checking if the perturbation follows the rules by using agents to extract definitive statements from the API documentation and developing return values that would fit with the client codebase that are modified and unconventional but stay within the bounds of the definitions. We would measure the amount of bugs we find and how long it takes to run this process.

2. **How will you be selecting target APIs? What is the systematic process? For example, are you only going to be targeting ordering issues? Do you have plans to extend and how?**

   We would be selecting target APIs based on what APIs are called by the client code. The agent, as it is analyzing the API documentation, would determine if the API call has a non-deterministic scope. We are not limited by targeting a particular type of issue, but ordering is a good place to start for us.

3. **What projects are you going to be running on? How will you obtain them (i.e., what systematic process will you follow)? Will there be intended overlap with prior work subjects?**

   We will be using the same client codebases that the NonDex paper uses. These codebases

are open-source. We will be using open-source LLMs as well. This heavily overlaps with the paper “NonDex: a tool for detecting and debugging wrong assumptions on Java API specifications”.

Progress

We’ve looked through supporting papers, primarily focusing on “NonDex: a tool for detecting and debugging wrong assumptions on Java API specifications”. We decided to use the same client projects that were used in NonDex: Checkstyle, Apache Jackrabbit Oak, JGraphT, and others. We have decided to use an open-source LLM (GLM-5.3, Deepseek V4, Kimi K3) on the TACC machines as a way to read specifications for APIs the client projects use and generate modified versions of the functions/output. Based on the API calls used in the client projects, we could investigate the specifications for that particular method within those APIs. However, we can start with APIs that NonDex currently covers such as Object hashCode(), HashMap keySet(), values(), entrySet(), ConcurrentHashMap elements(), values(), etc.

The evaluation guidelines for our project would revolve around how much more progress we made relative to the results in NonDex. The original methodology required manually creating a modified version of the API call and using NonDex to replace these calls at runtime. This would take a long time and possibly could hinder progress. Our metrics for success are driven by how many more non-deterministic caused bugs we can find using our LLM-based methodology relative to the paper’s manual implementation of NonDex. Additionally, we could report how long it took to run our agentic workflow from start to finish and make these changes.

The agentic workflow we’ve generally come up with:

1. A script identifies all the API calls executed for a specific test.
2. For each API, an agent searches for and reads the associated documentation and determines if it is nondeterministically specified. If so, continue to step 3. Otherwise, move to the next API call.
3. An agent uses the current implementation of the API as a starting point, but legally alters its functionality with a different implementation directly addressing the nondeterministic specification. Trigger NonDex on the test with modified function behavior.

Dividing Work

This part of the project was making decisions and identifying goals. We did this part of the project together through discussions.

Reiterate Goals

Our goals for checkpoint 1 were to select the target APIs and client projects, define the evaluation guidelines, and build an initial idea for the agentic workflow that identifies permitted ordering variations and supporting documentation passages. We met each goal besides fully isolating each target API. We've identified that our work will be an extension to NonDex and the "Detecting Assumptions on Deterministic Implementations of Non-deterministic Specifications" paper, focused on automating the manual effort. For now, we have decided to at least target the same APIs as written in this paper and the same target projects but we can expand our scope.

Next Steps

Our next steps include generating and validating perturbation wrappers for the selected APIs, integrating them with a test runner, and collecting initial failures from client projects as listed in our proposal. However, these results may not be feasible in time. Before we can do this, we need to first set up the environment we can use to experiment. This involves setting up the LLMs on TACC machines. Then, we will investigate NonDex to replace API method calls at runtime and understand exactly how to allow the LLM to dynamically input alternative method implementations. Aryan will work on setting up one of the open-source LLMs on the TACC machines while Hirsch will work on investigating NonDex and altering it.